



Spring DATA JPA

- ♦ CRUD Operations called Insert, Update, Retrive and Delete Opearitions are common across all DAOs. The only difference is domain types you are interacting with.
- ♦ Spring Data internally provides the implementation for all such common methods. So developer is not responsible for writing implementation for these common methods.

Ex:

```
public interface CustomerDAO extends JpaRepository<Customer, Integer>{  
    //Empty  
}
```

- ♦ JpaRepository is parameterized and takes two parameters
 - Entity class
 - Primary Key type
- ♦ CrudRepository is the root for All the Dpring Data Repositories.
- ♦ CrudRepository has 12 Methods.
- ♦ PagingAndSortingRepository extends CrudRepository
- ♦ PagingAndSortingRepository has 2 Methods.
- ♦ JpaRepository extends PagingAndSortingRepository.
- ♦ JpaRepository has 12 methods and 14 methods will be inherited.
- ♦ Your DAO interface which is extending JpaRepository can use all these methods for performing common persistence operations such as saving, deleting, finding

Methods of PagingAndSortingRepository

1.	Page<T> findAll(Pageable pageable)
2.	Iterable<T> findAll(Sort sort)



Methods of CrudRepository

1.	long count()
2.	void delete(T entity)
3.	void deleteAll()
4.	void deleteAll(Iterable<? extends T> entities)
5.	void deleteAllById(Iterable<? extends ID> ids)
6.	void deleteById(ID id)
7.	boolean existsById(ID id)
8.	Iterable<T> findAll()
9.	Iterable<T> findAllById(Iterable<ID> ids)
10.	Optional<T> findById(ID id)
11.	S save(S entity)
12.	Iterable<S> saveAll(Iterable<S> entities)

Methods of JpaRepository

1.	void deleteAllByIdInBatch(Iterable<ID> ids)
2.	void deleteAllInBatch()
3.	void deleteAllInBatch(Iterable<T> entities)
4.	List<T> findAll()
5.	List<T> findAll(Sort sort)
6.	List<T> findAllById(Iterable<ID> ids)
7.	void flush()
8.	T getById(ID id)
9.	T getOne(ID id)
10.	List<S> saveAll(Iterable<S> entities)
11.	List<S> saveAllAndFlush(Iterable<S> entities)
12.	S saveAndFlush(S entity)



demo1-data-jpa: Files required

1. MyBootApplication.java	
2. Customer.java	
3. CustomerDAO.java	
4. application.properties	
5. pom.xml	

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.Arrays;
import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        System.out.println(custDAO.getClass());

        // 1.save() method - inserting
        Customer cust1 = new Customer("bbb", "ccc@jlc", 222, "Blore");
        custDAO.save(cust1);
        System.out.println(cust1.getCid());

        //2. saveAndFlush() - A inserts if P.K not given
        Customer cust2 = new Customer("ccc", "ccc@jlc", 333, "Blore");
        custDAO.saveAndFlush(cust2);
```



//3. saveAndFlush() - B) updates if P.K given and present

```
Customer cust3 = new Customer(11,"bbb", "bbb@jlc", 222, "Blore");  
custDAO.saveAndFlush(cust3);
```

//4. saveAndFlush() -C) Inserts if P.K given and not present

```
Customer cust4 = new Customer(99,"raheem", "raheem@jlc",1234567, "Hyd");  
custDAO.saveAndFlush(cust4);
```

//5. findAll

```
List<Customer> mylist1 = custDAO.findAll();  
mylist1.stream().forEach(System.out::println);
```

//6. findAllById

```
List<Integer> myIds1 = Arrays.asList(11,12,13,14);  
List<Customer> mylist2 = custDAO.findAllById(myIds1);  
mylist2.stream().forEach(System.out::println);
```

//7. count

```
long count = custDAO.count();  
System.out.println(count);
```

//8. saveAll

```
Customer cust5 = new Customer("pp", "pp@jlc", 222, "Blore");  
Customer cust6 = new Customer("qq", "qq@jlc", 222, "Blore");  
Customer cust7 = new Customer("rr", "rr@jlc", 222, "Blore");  
List<Customer> custList = Arrays.asList(cust5,cust6,cust7);  
custDAO.saveAll(custList);
```

//9. findById

```
Optional<Customer> myOps = custDAO.findById(77);  
myOps.ifPresent(System.out::println);
```

//10. getOne

```
Customer cust8 = custDAO.getOne(77);  
System.out.println(cust8);
```

//11. getById

```
Customer cust9 = custDAO.getById(77);  
System.out.println(cust9);
```

//12. existsById

```
System.out.println(custDAO.existsById(77));  
System.out.println(custDAO.existsById(77));
```



```
//13. delete(entity)
Optional<Customer> myOps1 = custDAO.findById(502);
myOps1.ifPresent(mycust -> custDAO.delete(mycust));

//13. delete(entity)
if(myOps.isPresent()) {
    Customer cust = myOps.get();
    custDAO.delete(cust);
}

//14. deleteById
custDAO.deleteById(505);

//15. deleteAll
Customer cust10 = custDAO.findById(501);
Customer cust11 = custDAO.findById(503);
Customer cust12 = custDAO.findById(504);

List<Customer> custList1 = Arrays.asList(cust10,cust11,cust12);
custDAO.deleteAll(custList1);

//16. deleteAllInBatch
custDAO.deleteAllInBatch(custList1);

//17. deleteAllById
List<Integer> myIds = Arrays.asList(10,11,12);
custDAO.deleteAllById(myIds);

//18. deleteAllByIdInBatch
custDAO.deleteAllByIdInBatch(myIds);

//19. deleteAll
custDAO.deleteAll();

//20. deleteAllInBatch
custDAO.deleteAllInBatch();

System.out.println("Done!!!");
}

public static void main(String[] args) {
    SpringApplication.run(MyBootApplication.class, args);
}
}
```



2) Customer.java

```
package com.jlcindia.springboot;

import javax.persistence.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Entity
@Table(name="mycustomers")
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="cid")
    private int cid;

    @Column(name="cname")
    private String cname;

    @Column(name="email")
    private String email;

    @Column(name="phone")
    private int phone;

    @Column(name="city")
    private String city;

    public Customer() {}
    public Customer(String cname, String email, int phone, String city) {
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
    }
    public Customer(int cid, String cname, String email, int phone, String city) {
        super();
        this.cid = cid;
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
    }
    //Setters and Getters
    //toString()
}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import org.springframework.data.jpa.repository.JpaRepository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO extends JpaRepository<Customer, Integer>{

}
```

4)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```

#JPA Properties

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n
```

5)pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo1-data-jpa</artifactId>
<version>1.0</version>
<name>demo1-data-jpa</name>
<description>This is Data JPA Example</description>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>
```



```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Demo2-data-jpa: Files required

1. MyBootApplication.java	Updated in Demo2
2. CustomerService.java	Newly added in Demo2
3. CustomerServiceImpl.java	Newly added in Demo2
4. Customer.java	Same as Demo1
5. CustomerDAO.java	Same as Demo1
6. application.properties	Same as Demo1
7. pom.xml	Same as Demo1

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```



```
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerService customerService;

    @Override
    public void run(String... args) throws Exception {

        customerService.findCustomerById(77).ifPresent(System.out::println);

        customerService.findCustomerById(7).ifPresent(System.out::println);

        customerService.getCustomerById(7).ifPresent(System.out::println);

        customerService.getCustomerById(77).ifPresent(System.out::println);
    }

    public static void main(String[] args) {
        SpringApplication.run(MyBootApplication.class, args);
    }
}
```

2) CustomerService.java

```
package com.jlcindia.springboot;

import java.util.Optional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public interface CustomerService {
    public Optional<Customer> findCustomerById(int cid);
    public Optional<Customer> getCustomerById(int cid);
}
```



3) CustomerServiceImpl.java

```
package com.jlcindia.springboot;

import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Service
@Transactional
public class CustomerServiceImpl implements CustomerService{

    @Autowired
    CustomerDAO customerDAO;

    @Override
    @Transactional(isolation = Isolation.READ_COMMITTED,propagation =
    Propagation.REQUIRED)
    public Optional<Customer> findCustomerById(int cid) {
        Optional<Customer> custOpts= customerDAO.findById(cid);

        return custOpts;
    }

    @Override
    @Transactional(isolation = Isolation.READ_COMMITTED,propagation =
    Propagation.REQUIRED)
    public Optional<Customer> getCustomerById(int cid) {

        Optional<Customer> custOpts=Optional.empty();
        try {
            Customer cust = customerDAO.getById(cid);
            cust.getCname();
            custOpts=Optional.of(cust);
        }catch(Exception ex) { }

        return custOpts;
    }
}
```